

CardStack

Design D0: High-Level System Design

1. Title, Goal Statement, and Conventions

Goal Statement

CardStack is a payment system that allows consumers to complete online purchases quickly while automatically selecting the user's credit card that provides the best rewards for the purchase category.

Diagram Conventions

- **Rounded rectangles** represent CardStack components developed by the team.
- **Gray/external boxes** represent external systems that CardStack depends on.
- **Solid arrows** represent data or control being sent between components.
- **Interface IDs (I1, I2, etc.)** identify the interfaces described in the Interface Specification Table.
- **Directional arrows** indicate the direction of information flow.

Basic Input

The basic input to CardStack is a **checkout/payment request** containing information such as the purchase amount, merchant, transaction details, and authenticated consumer information.

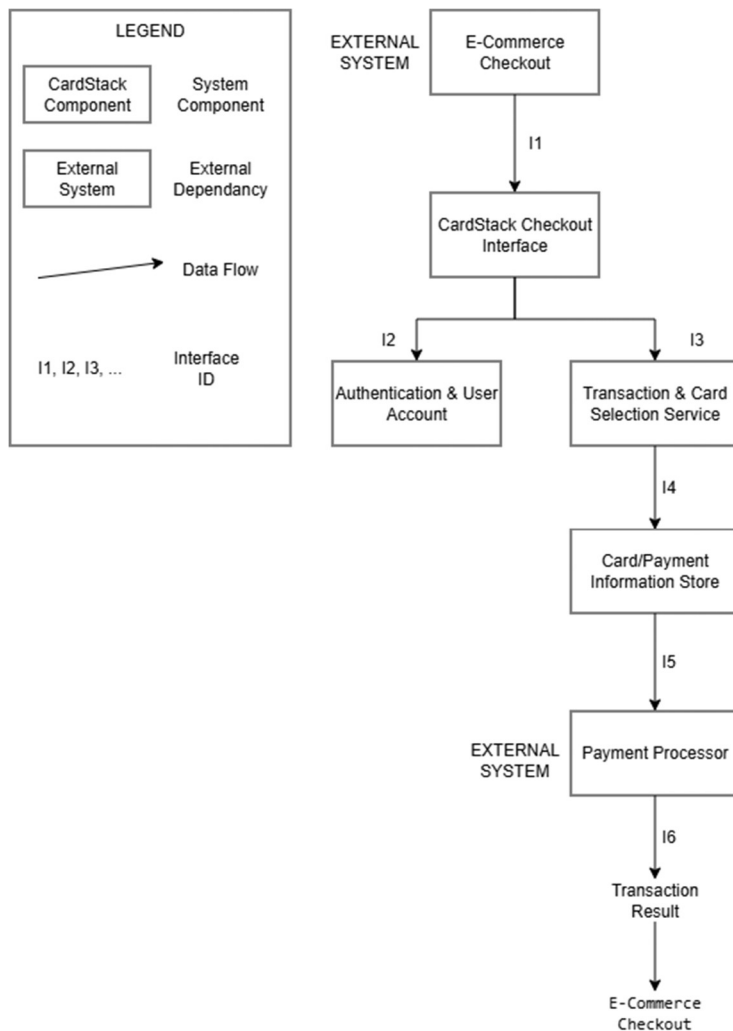
Basic Output

The basic output is a **payment result** indicating whether the purchase succeeded or failed, along with the appropriate transaction status returned to the ecommerce website.

2. Block Diagram — D0

Components

1. **E-Commerce Checkout**
2. **CardStack Checkout Interface**
3. **Authentication & User Account**
4. **Transaction & Card Selection Service**
5. **Card/Payment Information Store**
6. **Payment Processor**



3. Component responsibility table

Component	Responsibility	Interfaces In	Interfaces Out	Primary Owner
E-Commerce Checkout	Provides the customer's checkout experience and sends the purchase request to CardStack.	16	I1	Luke
CardStack Checkout Interface	Receives checkout requests, presents payment information to the consumer, and communicates the payment result back to the ecommerce site.	I1, I6	I2, I3	Luke
Authentication & User Account	Verifies the consumer's identity and provides access to the consumer's saved CardStack information.	I2	I2	Maya
Transaction & Card Selection Service	Determines which saved credit card should be used for the transaction and coordinates payment processing.	I3, I4	I4, I5	Maya
Card/Payment Information Store	Securely stores the consumer's saved payment-card information and associated reward/category information.	I4	I4	Maya
Payment Processor	Processes the requested payment using the selected credit card and returns a transaction result.	I5	I6	External System

4. Interface specification table

ID	Interface	Inputs	Outputs	Data Format	Protocol	Error Handling
I1	Ecommerce → Checkout Interface	Purchase amount, merchant information, cart/order ID	Checkout session/request confirmation	JSON	HTTPS	Invalid request is rejected and an error is returned to ecommerce checkout Invalid or expired authentication requires the consumer to authenticate again
I2	Checkout Interface ↔ Authentication	User/session information	Authentication status and user ID	JSON	HTTPS	Invalid transaction data returns an error to the Checkout Interface
I3	Checkout Interface → Transaction Service	Authenticated user ID, transaction amount, merchant/category information	Transaction request acknowledgment	JSON	HTTPS	Missing or unavailable card data causes transaction failure and user notification
I4	Transaction Service ↔ Card Information Store	User ID, transaction category, card information request	Eligible card information and reward/category data	JSON	HTTPS	Declined/failed payment is returned to
I5	Transaction Service →	Selected payment token, transaction	Payment authorization result	JSON	HTTPS	

	Payment Processor	amount, merchant information				Transaction Service for error handling Failed payment returns failure status and allows the user to select another payment method
I6	Checkout Interface ↔ Ecommerce Checkout	Payment status, transaction ID, failure information if applicable	Updated order/payment status	JSON	HTTPS	

Example Payload

Ecommerce website could send CardStacker a payment request like:

```
{
  "orderId": "ORD-10482",
  "amount": 89.99,
  "currency": "USD",
  "merchant": "ExampleStore",
  "category": "Clothing"
}
```

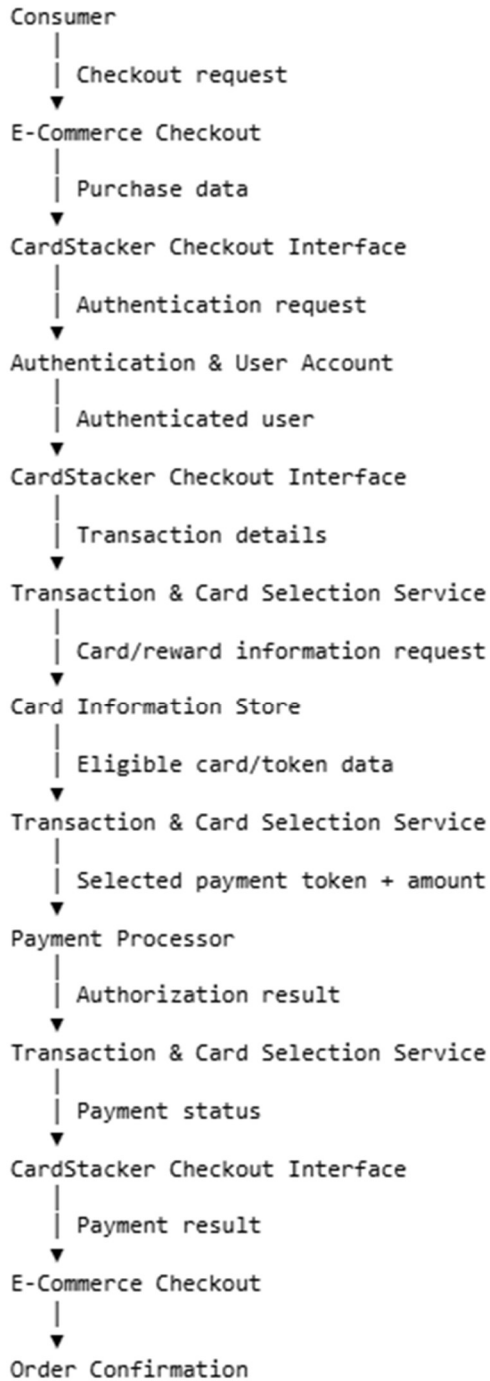
A successful payment response:

```
{
  "orderId": "ORD-10482",
  "status": "approved",
  "transactionId": "TX-58291"
}
```

A failed transaction returns:

```
{  
  "orderId": "ORD-10482",  
  "status": "declined",  
  "reason": "Payment declined"  
}
```

5. Data flow diagram



6. Architecture pattern and justification

Architecture Pattern

CardStack uses a combination of **client-server** and **layered architecture** patterns.

Client-Server

The ecommerce checkout and CardStack checkout interface act as clients that communicate with CardStack's backend services.

The backend is responsible for authentication, transaction coordination, card selection, and communication with external payment infrastructure.

Layered Architecture

The CardStack system can be divided into several high-level layers:

Presentation Layer

- CardStack Checkout Interface

Application/Service Layer

- Transaction & Card Selection Service
- Authentication & User Account

Data Layer

- Card/Payment Information Store

External Services

- Ecommerce Checkout
- Payment Processor

This separation allows each major responsibility to remain independent at the D0 level.

Pattern Justification

1. Fit to the Problem

Client-server architecture fits CardStack because consumers interact with CardStack through an ecommerce website while the sensitive payment and account operations are handled by backend services.

Layering also separates the user-facing checkout experience from transaction processing and stored payment information.

2. Team Skills

The team can divide development responsibilities between the user interface, backend services, authentication, and data storage. This allows multiple team members to work on separate components while maintaining defined interfaces between them.

3. Performance and Timing

The system has a requirement that a purchase should complete within **5 seconds**. Keeping the checkout interface and transaction services separated allows the payment process to be handled without requiring the ecommerce website to directly manage CardStack's internal operations.

The number of network requests and external payment dependencies should be minimized during later design stages to help meet this requirement.

4. Scalability

A client-server architecture allows additional ecommerce merchants and consumers to use the CardStack service without requiring CardStack logic to be installed separately on each merchant's system.

Individual backend components can also be expanded independently as system usage grows.

5. Hardware Constraints

CardStack does not require specialized hardware. The system primarily operates through web clients, backend servers, databases, and external payment infrastructure. Therefore, a client-server/layered architecture avoids unnecessary hardware requirements.

Rejected Pattern: Embedded Sensor-Actuator

The embedded sensor-actuator pattern was rejected because CardStack does not depend on physical sensors, actuators, or specialized hardware. The system's primary interactions occur between software components and external web/payment services.

7. Decision log

Decision	Alternatives Considered	Chosen Option	Reason
D-01: How CardStacker integrates with ecommerce sites	Direct modification of merchant payment systems; CardStacker checkout interface	CardStacker checkout interface	A defined checkout interface allows merchants to integrate CardStacker without requiring CardStacker to control the merchant's entire checkout system.
D-02: How payment information is handled	Store raw credit card numbers; store encrypted/tokenized payment information	Tokenized/encrypted payment information	The stakeholder requirements state that credit card PANs cannot be stored in plain text. Tokenized or encrypted information reduces exposure of sensitive payment information.
D-03: How the system selects a card	Consumer manually chooses a card; CardStacker automatically selects a card	Automatic selection based on purchase category and stored reward information	US-02 requires CardStacker to select the card that provides the best benefits for the purchase category.
D-04: Architecture pattern	Microservices; client-server/layered architecture	Client-server with layered architecture	The system needs separation between the checkout interface, backend transaction processing, authentication, and

Decision	Alternatives Considered	Chosen Option	Reason
			data storage without requiring the additional complexity of a full microservices architecture.
D-05: Payment failure handling	End the transaction after a declined card; return failure and allow another payment method	Return failure to checkout and allow an alternative payment method	UC-01.2 explicitly requires CardStacker to notify the consumer of a card failure and allow another payment method.